

UNIDAD 1 — INGENIERÍA DE SOFTWARE

Concepto de software: El software es el conjunto de programas, datos y documentación que permiten que una computadora funcione y resuelva problemas.

Fallas del hardware (curva en bañera): El hardware presenta muchas fallas al principio por defectos de fabricación (mortalidad infantil), luego la tasa de fallas disminuye y se mantiene estable durante su vida útil, y finalmente vuelve a aumentar debido al desgaste físico de los componentes.

Fallas del software: El software no se desgasta físicamente. Las fallas aparecen principalmente cuando se realizan cambios que no se testean bien, arquitectura mal hecha, mantenimiento o se agregan nuevas funcionalidades. Cada modificación puede introducir errores o efectos colaterales. Se termina convirtiendo en un Monolito (un “frankenstein” que tiene muchas cosas, se hace muy complejo y no termina conviniendo)

Ingeniería de Software: Es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software. También incluye el estudio de estos enfoques.

Se estructura en varias capas:

- 1) Compromiso con la calidad
- 2) El proceso, que organiza el trabajo
- 3) Los métodos, que son las técnicas utilizadas para desarrollar software.
- 4) Las herramientas que ayudan a aplicar los métodos y procesos.

La diferencia entre sistema y software: el software es un pedazo de un sistema. El sistema es lo que engloba el “todo”, hardware, requerimientos, etc.

Modelos de desarrollo:

- 1) **Modelo en cascada:** Es un modelo de desarrollo secuencial en el que cada fase del proyecto debe completarse antes de pasar a la siguiente.
- 2) **Modelo en espiral:** Es un modelo iterativo propuesto por Barry Boehm que combina desarrollo incremental con análisis de riesgos.
- 3) **Metodologías ágiles:** Surgen para aumentar la flexibilidad y la adaptación al cambio, promoviendo la colaboración con el cliente y las entregas frecuentes.
- 4) **CMM y CMMI:** Son modelos de madurez que permiten evaluar y mejorar los procesos de desarrollo de software dentro de una organización.
- 5) **DevOps:** Es un enfoque que integra desarrollo y operaciones con el objetivo de lograr entregas más rápidas, confiables y automatizadas.

Ambiente de desarrollo: Es el entorno utilizado por los programadores para crear, modificar y probar el software antes de pasarlo a testing o producción. Allí se escriben los cambios, se instalan herramientas, se hacen pruebas unitarias y se verifican configuraciones sin afectar a los usuarios reales. Es un ambiente flexible, donde se pueden cometer errores o hacer cambios frecuentes porque no impacta directamente en el sistema final. Por ejemplo, allí se pueden cambiar contraseñas de servicios de correo, agregar claves API o instalar nuevos paquetes de software antes de llevar esos cambios a otros entornos.

3 principales diferencias entre modelo tradicional y ágil: El modelo tradicional trabaja de forma secuencial, con muchas etapas fijas y poca posibilidad de cambio una vez iniciado el proyecto. El modelo ágil trabaja de forma iterativa, haciendo entregas parciales y permitiendo adaptarse a cambios. Además, en el modelo tradicional el cliente participa poco, mientras que en ágil participa constantemente.

Ejemplo concreto donde conviene una metodología tradicional y no una ágil: Un software para controlar un reactor nuclear o un sistema bancario crítico conviene desarrollarlo con una metodología tradicional, porque los requisitos son muy estrictos, no deben cambiar y se necesita mucha documentación y control antes de implementar. Las metodologías ágiles son más útiles cuando los requisitos cambian frecuentemente.

Variables de gestión de proyectos: Se refiere a tiempo, costo y alcance. Si una cambia, las otras también se ven afectadas. Por ejemplo, si se quiere terminar antes, puede aumentar el costo o reducirse el alcance.

SWEBOK: Es una guía que describe el conjunto de conocimientos generalmente aceptados en la disciplina de la ingeniería de software. Describe qué temas debe conocer un ingeniero de software, como requisitos, diseño, pruebas, mantenimiento, calidad y gestión.

Objetivos:

- Promover una visión consistente de la ingeniería de software
- Definir los límites de la disciplina
- Caracterizar sus contenidos
- Proporcionar una base de conocimiento común
- Servir como referencia para planes de estudio y certificaciones profesionales.

Aspectos:

- Educación:** El SWEBOK se utiliza como marco para diseñar programas de estudio en ingeniería de software. Define qué contenidos deben enseñarse para formar profesionales competentes.
- Desarrollo de software:** Las empresas lo usan para identificar las áreas de conocimiento necesarias en los proyectos. Sirve para planificar, asignar recursos y capacitar al equipo.
- Certificación y acreditación:** Se utiliza como referencia para evaluar el nivel profesional de ingenieros de software o validar programas académicos.
- Mejora de procesos:** Permite detectar debilidades en la forma de desarrollar software y aplicar mejores prácticas para aumentar la calidad y eficiencia.

Áreas de conocimiento del SWEBOK: Incluyen requisitos, diseño, construcción, pruebas, mantenimiento, gestión de configuración, gestión de ingeniería de software, procesos de ingeniería, modelos y métodos, calidad, práctica profesional, economía, fundamentos informáticos, fundamentos matemáticos y fundamentos de ingeniería.

Aplicación del SWEBOK en la práctica: Se utiliza en la educación para diseñar programas académicos, en las empresas para organizar proyectos y desarrollar competencias, en certificaciones profesionales y en la mejora de procesos para aumentar la calidad y eficiencia del desarrollo de software.

Donde se usa la Ingeniería de Software

- 1) **Inteligencia Artificial y Machine Learning:** Están impactando significativamente la ingeniería de software mediante la automatización de pruebas, el análisis de datos, la optimización de procesos y la automatización de tareas repetitivas.
- 2) **Arquitecturas emergentes:** Incluyen microservicios y contenedores, que facilitan la escalabilidad y flexibilidad en el desarrollo y despliegue de sistemas.
- 3) **Computación en la nube:** Permite desarrollar y desplegar software en infraestructuras distribuidas y escalables.
- 4) **Computación cuántica:** Representa una tecnología emergente que puede transformar ciertos tipos de aplicaciones y algoritmos.
- 5) **Realidad aumentada:** Influye en la forma en que se diseñan interfaces y experiencias de usuario.

Seguridad y privacidad: Deben integrarse desde el inicio del ciclo de vida del software para garantizar la protección de datos y sistemas.

Metodologías ágiles dominantes: Scrum, Kanban y Extreme Programming promueven la flexibilidad, la colaboración y la entrega continua de valor.

Transformación digital: La ingeniería de software permite a las organizaciones adaptarse a entornos digitales mediante el desarrollo de aplicaciones, sistemas y plataformas tecnológicas.

DevOps y automatización: La integración entre desarrollo y operaciones, junto con prácticas como integración continua y entrega continua, permite despliegues más rápidos y confiables.

¿cómo contribuyen a la colaboración de equipos de desarrollo y operaciones?: DevOps mejora la colaboración porque ambos equipos trabajan juntos desde el inicio del proyecto. Comparten herramientas, automatizan tareas y coordinan despliegues y mantenimiento, evitando conflictos entre “desarrollar” y “operar”.

PREGUNTAS:

1. Que es ingenieria de software
2. es base al grafico de tiempo de vida de hardware y software, explique
3. que quiere decir cada movimiento de tasa de fallos
4. Cuales son las capas de ingeniería de software
5. Que es SWEBOK
6. Cuales son las ventajas y desventajas de SWEBOK
7. De los 5 objetivos de SWEBOK explique 2
8. Cuales son los 4 pilares de la ing de software
9. Cual es la aplicación del swbok en la práctica de la ingeniería de sw. Ejemplifique 2
10. Desarrolle ing de sw con iA y aprendizaje automático
11. ing de sw con seguridad y privacidad
12. ing de sw con DevOps y automatización

UNIDAD 2 — Operaciones de Ingeniería de Software

Procesos de operación en Ingeniería de Software: son todas las actividades relacionadas con la gestión, mantenimiento y soporte continuo de los sistemas una vez que ya fueron desarrollados e implementados.

Objetivo: es garantizar que el software funcione correctamente, cumpla con los requisitos del usuario y pueda adaptarse a cambios del entorno operativo.

Actividades principales de operación

Despliegue del software: Consiste en instalar el software en el entorno donde va a funcionar. Puede incluir configuración de servidores, bases de datos y otros componentes necesarios.

Monitoreo del sistema: Se controla el rendimiento del software en producción para detectar problemas y asegurar un funcionamiento óptimo. Métricas típicas:

- Uso de recursos

- Tiempo de respuesta

- Disponibilidad

Gestión de incidencias: Consiste en detectar un problema, registrarlo, priorizarlo según su gravedad, asignarlo a una persona o equipo y resolverlo. Luego se documenta la solución para evitar que vuelva a ocurrir.

Software de gestión de incidencias: Es una herramienta que permite registrar, organizar, priorizar, asignar y resolver incidentes o problemas que ocurren durante la operación de un sistema de software. Su objetivo es mejorar el soporte, reducir tiempos de respuesta y asegurar la continuidad del servicio.

Ejemplo: Jira Service Management: Es un software de gestión de incidencias muy usado por equipos IT y de desarrollo. Permite crear tickets, asignarlos a responsables, establecer prioridades, hacer seguimiento del estado de cada incidente y mantener trazabilidad de todo el proceso hasta su resolución.

Riesgos comunes en la gestión de un proyecto y qué variables se tienen en cuenta para mitigarlos: Algunos riesgos comunes son falta de tiempo, presupuesto insuficiente, requisitos poco claros y fallas técnicas. Para mitigarlos se tienen en cuenta variables como tiempo, costo, alcance, recursos y probabilidad de ocurrencia del riesgo.

Actualizaciones y parches: Los sistemas de software requieren actualizaciones periódicas (Incluye planificación, pruebas e implementación) para:

- Corregir errores

- Agregar funcionalidades

- Mejorar seguridad y rendimiento

Respaldos y recuperación: Se realizan copias de seguridad periódicas para garantizar la disponibilidad e integridad de los datos ante fallos o desastres. También implica:

- Planes de recuperación

- Procedimientos ante pérdida de información

Optimización del rendimiento: Se busca mejorar la experiencia del usuario optimizando:

- Consultas a bases de datos

- Algoritmos

- Infraestructura

- Instalación de paquetes de software

¿Qué implica la integridad en todos los entornos de software? Que esté todo relacionado (que lo que digo en un ambiente tenga coherencia en otro).

Instalación de paquetes de software: Antes de instalar software es necesario preparar el sistema y verificar información importante. Esto puede variar según el sistema operativo

Aspectos clave:

1. Revisar notas de versión
2. Verificar compatibilidad de hardware y sistema operativo
3. Descargar software desde fuentes oficiales
4. Validar integridad del paquete (firmas digitales o checksums)
5. Contar con privilegios de administrador si es necesario
6. Configurar opciones durante la instalación
7. Mantener actualizaciones periódicas
8. Consultar documentación oficial

scripting: Permite automatizar tareas repetitivas para ahorrar tiempo y reducir errores en la implementación del software en múltiples sistemas.

Herramientas de gestión de configuración: Permiten definir y mantener automáticamente el estado deseado del sistema.

Contenedores: Permiten encapsular aplicaciones con todas sus dependencias para facilitar despliegue y escalabilidad. Ejemplo: Docker. Ventajas: Los contenedores permiten ejecutar una aplicación con todas sus dependencias en cualquier entorno. Portabilidad, Consistencia, Implementación rápida

Imágenes de sistema: Son snapshots del sistema operativo con software ya instalado y configurado. Permiten desplegar sistemas nuevos rápidamente.

Gestores de paquetes y repositorios: Simplifican y automatizan la instalación de software.

Pruebas y solución de problemas: Las pruebas son fundamentales para garantizar que los scripts y configuraciones funcionen correctamente.

Tipos de pruebas:

Pruebas unitarias:

- Verificación de sintaxis
- Pruebas en entornos de desarrollo

Pruebas de integración

- Simulación de escenarios reales
- Pruebas end-to-end

Registro y monitoreo

- Generación de logs detallados
- Monitoreo continuo para detectar problemas antes que impacten al usuario

Solución de problemas:

- Identificación de errores mediante logs
- Revisión de cambios recientes
- Pruebas de diagnóstico para aislar fallos

Mejora continua

- Retroalimentación de usuarios
- Revisión periódica de scripts
- Actualización para evitar problemas de seguridad o compatibilidad

Documentación: Debe ser clara y concisa e incluir:

- Procedimientos de instalación

Configuración
Resolución de problemas
Notas de versión
Performance, Reliability y Balanceo de carga

Un buen software siempre debe evita sobrecarga, mejora disponibilidad, permite escalabilidad

Performance: Mide la velocidad y eficiencia del sistema. Objetivo:

Mejorar tiempos de respuesta
Optimizar recursos

Fiabilidad: Capacidad del sistema de funcionar sin fallos y mantener disponibilidad.

Sistemas redundantes
Planes de recuperación

Balanceo de carga: Distribuye solicitudes entre servidores para mejorar rendimiento y estabilidad.

Plan de operaciones: Es un documento que describe cómo se gestionan las actividades del software en distintos entornos.

Disponibilidad de software: Es la capacidad del sistema de estar accesible cuando los usuarios lo necesitan.

Conceptos clave:

- 1) Tiempo de actividad (uptime)
- 2) Tiempo de inactividad (downtime)
- 3) Resiliencia (recuperarse rápido ante fallos)

Continuidad del servicio: Implica que el sistema siga funcionando incluso en situaciones críticas. Incluye:

- 1) Plan de continuidad del negocio (BCP) **hay q expandir la definicion y ejemplos**
- 2) Recuperación ante desastres (DR) **hay q expandir la definicion y ejemplos**
- 3) Pruebas y simulacros

Niveles de servicio (SLA): Es un acuerdo entre proveedor y cliente que define qué nivel de servicio debe cumplir el sistema, por ejemplo disponibilidad, tiempo de respuesta y tiempo de resolución.

- 1) Disponibilidad
- 2) Tiempo de respuesta
- 3) Tiempo de resolución

Dimensionamiento: Es el proceso de determinar qué recursos necesita el sistema para funcionar correctamente según la carga. Incluye:

1. Análisis de carga
2. Estimación de CPU, memoria, red, almacenamiento
3. Planificación de capacidad con margen para picos

Gestión de capacidad: Consiste en monitorear y optimizar el uso de recursos para satisfacer necesidades actuales y futuras. Actividades:

- 1) Monitoreo del rendimiento
- 2) Optimización de recursos
- 3) Planificación
- 4) Escalabilidad

Escalabilidad: Capacidad del sistema de crecer según demanda.

Escalabilidad horizontal: Agregar más servidores y Distribuir carga

Escalabilidad vertical: Aumentar recursos de un servidor (RAM, CPU)

Scripting para automatización de software: El scripting se utiliza para automatizar tareas repetitivas o técnicas dentro del ciclo de operación del software, reduciendo tiempo, esfuerzo manual y errores.

Puede aplicarse a la automatización de pruebas de software

Tareas de administración del sistema (configuración de servidores, instalación de aplicaciones, gestión de usuarios y permisos)

Procesamiento de datos. (ETL, es decir, extracción, transformación y carga de datos desde distintas fuentes hacia bases de datos)

Entrega de operaciones de software: Es el proceso de preparación y puesta en marcha de un sistema de software en un entorno operativo real, listo para ser utilizado por los usuarios finales. No se trata solamente de “instalar” el software, sino de dejarlo funcionando de forma correcta, verificada y aceptada para su uso real.

Sistema de software: Está compuesto por aplicaciones o programas, datos, interfaces de usuario, componentes de infraestructura y reglas de negocio con su lógica de aplicación. Es decir, el sistema es un conjunto más amplio que el simple código.

Pruebas operativas: Son las pruebas que se realizan para comprobar que el sistema funciona correctamente en el entorno operativo real para el cual fue diseñado. Son situaciones de uso típicas y se verifica que el software responda adecuadamente a las solicitudes de los usuarios, procese los datos de forma eficiente y mantenga la integridad de los datos. (performance, disponibilidad, capacidad y monitoreo) Ejemplos:

Pruebas de rendimiento,

Pruebas de carga,

Pruebas de estabilidad,

Pruebas de disponibilidad.

Verificación: Es el proceso de asegurarse de que el software entregado cumple con los requisitos y especificaciones establecidos durante la fase de desarrollo. Se comparan los resultados obtenidos en las pruebas operativas con los criterios de aceptación definidos previamente, para determinar si el software funciona como se esperaba. La verificación puede incluir:

Comprobación de funciones específicas,

Corrección de errores o defectos encontrados,

Confirmación de cumplimiento de estándares de calidad y desempeño.

Elementos que se usan en la verificación

Artefactos importantes que se usan para verificar:

Documento de Requisitos del Software (SRS): Documento que especifica los requisitos funcionales y no funcionales del software. Especifica los requisitos funcionales, no funcionales, restricciones y criterios de calidad. del software

Matriz de trazabilidad de requisitos: Muestra la relación entre los requisitos y los distintos elementos del sistema. Contiene la lista de requisitos y su vínculo con casos de uso, pruebas y componentes. Debe actualizarse cuando cambian los requisitos o se agregan elementos nuevos

Casos de uso y diagramas de flujo: Representan procesos, interacciones y comportamiento esperado del sistema.

Prototipos: Versiones preliminares del sistema que ayudan a validar comportamiento, funcionalidades o diseño.

Informes y minutas de pruebas: Documentos que registran qué se probó, cómo se probó y qué resultados se obtuvieron.

Aceptación: Se produce cuando el cliente o el usuario final confirma que el sistema entregado cumple con sus expectativas y con los requisitos comerciales. Es el momento en el que, desde el punto de vista del negocio, se considera que el software está correcto para ser usado.

Pruebas de aceptación del usuario (UAT): Son pruebas realizadas por el cliente en un entorno similar al de producción para verificar que todas las características y funcionalidades requeridas estén presentes y funcionen correctamente.

Migración de datos: Es el proceso de mover datos entre distintos tipos de bases de datos, sistemas de gestión de datos, aplicaciones o plataformas tecnológicas. La presentación la destaca como un componente crucial en proyectos de desarrollo, actualización de sistemas, consolidación de bases de datos, cambio de proveedor de servicios y otras situaciones similares.

Aspectos a considerar en la migración de datos

- Planificación

- Clasificación de datos

- Selección de herramientas y tecnologías

- Análisis de limpieza de datos y estandarización

- Mapeo de datos

- Pruebas: Hay que revisar constantemente que no se haya migrado con algún error.

- Validación: Hay distintos procesos que ayudan a la validación de que se haya pasado sin errores, checksums, etc.

- Consideraciones de seguridad

- Ética y cumplimiento

Se migran no sólo los datos, sino los niveles de seguridad que tienen todos los usuarios, el objetivo es que sea la misma información pero en otra arquitectura

Herramientas: Se eligen las herramientas según los tipos de datos que tengamos, el volumen, el tiempo de inactividad permitido, la infraestructura y costos permitidos que tengamos

- Herramientas OnPremise:** Son herramientas locales, se utilizan con datos sensibles.

- Herramientas en la nube:** Son herramientas en la nube, son muy buenas ya que ofrecen escalabilidad automática sin la necesidad de infraestructura adicional (a un costo). Se usa cuando se requiere velocidad, no tener downtime en el servicio (que no se pause para hacer la migración)

- Herramientas híbridas:** Son herramientas que mezclan las dos, son buenas para cuando se tienen datos sensibles pero no se quiere perder la velocidad de la nube para los datos que no lo son

Gestión de cambios: 10 pasos para realizar una gestión de cambio

- Identificar la necesidad del cambio

- Analizar y evaluar la realización del cambio

- Evaluación de los riesgos

- Planificación

- Comunicación del cambio

- Implementación del cambio

- Evaluación y seguimiento del plan de trabajo

- Comunicación de los resultados a corto plazo

- Generar las acciones correctivas

- Adoptar el cambio en la cultura de la empresa

Problemas que pueden aparecer en una gestión de cambios

- Falta de planificación adecuada

- Falta de claridad sobre los objetivos del cambio

- Falta de recursos necesarios

- Riesgos asociados

- Resistencia al cambio

- Falta de capacitación

- Falta de comunicación

- Falta de participación

- Dificultad para implementar el cambio

- Comunicación inadecuada

- Expectativas poco realistas sobre el cambio

Retrasos en la implementación o problemas técnicos inesperados
Plazos irrealistas o promesas exageradas sobre los beneficios esperados
Desviación del plan de cambio
Cambios de alcance
Falta de seguimiento y evaluación
Ausencia de corrección de desviaciones

PREGUNTAS:

1. Explique el concepto de operaciones en ing de sw
2. Proceso de operación en ing de sw, que son?
3. De las actividades de las operaciones de ing de sw, desarrolle 4 (despliegue, mantenimiento, monitoreo)
4. identifique o mencione 4 métricas de monitoreo de sw
5. Que implica la integridad en todos los entornos de sw
6. Para que se utiliza un sw de gestión de incidencias
7. ejemplifica un sw del mercado actual
8. Que es un script de automatización
9. desarrolle 1 escenario en el que se utilice 1 script de automatización
10. que herramientas de sw me ayudan para el desarrollo del script de automatización (habia algunas
11. especiales, n8n, python, etc)
12. Describir que son contenedores
13. Proponga una arquitectura de dockets
14. Que son imagenes de sistemas vs gestores de paquetes
15. Diferencie y ejemplifique pruebas unitarias y pruebas de integración
16. Qué son las pruebas de extremo a extremo
17. A qué nos referimos cuando hablamos de monitoreo continuo, ejemplifica al menos 2 casos
18. Que son las pruebas unitarias
19. Que son pruebas operativas, mencione 2 ejemplos
20. Para que se utiliza la documentación de un software
21. Diferencia entre performance, fiabilidad y balanceo de carga
22. Para que me sirve un plan de operaciones
23. Desarrolle al menos 3 ambientes de ciclo de vida de desarrollo de software (pueden ser, desarrollo, testing,
24. etC)
25. A que nos referimos cuando hablamos de disponibilidad de sw y continuidad y niveles de servicio
26. Que es VCP (PLAN DE CONTINUIDAD DEL NEGOCIO)
27. Que es DR (RECUPERACION ANTE DESASTRE)
28. Niveles de servicio (SLA) desarrolle
29. A que nos referimos con el termino analisis de la carga
30. Desarrolle gestión de capacidad en it
31. Que es escalabilidad horizontal vs vertical
32. Explique la diferencia entre pruebas operativas, verificacion y aceptacion
33. En base a un sistema de sw, describa 3 elementos
34. Explique con argumentación técnica si el equipo de it debería tener acceso a un entorno de datos
35. productivos según lo visto en clase
36. Cual es la diferencia entre verificacion y aceptacion, ejemplifique
37. Que debemos tener en cuenta para una migración de datos, explique al menos 4 aspectos
38. Explique las ventajas de la gestion de cambios
39. De los 10 pasos de gestion de cambios, explique 3
40. Que problemas se pueden presentar en una gestion de cambios, explique al menos 4

UNIDAD 3 — PROCESO DE TERCERIZACIÓN

Tercerización / BPO (Business Process Outsourcing): Es la subcontratación de funciones o procesos de negocio a un proveedor de servicios, interno o externo a la empresa, que se supone más eficiente, eficaz o menos costoso que realizarlos dentro de la organización. La idea principal es delegar ciertas actividades para que la empresa pueda concentrarse en su negocio principal.

Objetivo: Reducir costos, aumentar eficiencia, acceder a tecnología o conocimientos especializados y permitir que la empresa se concentre en sus actividades principales.

Front Office: Son las actividades que tienen contacto directo con el cliente.

Incluye: Servicio al cliente, Ventas, Soporte técnico, Cobranza y reclamos, Recepción, Mesa de ayuda o Encuestas de satisfacción

Back Office: Son las actividades internas de la empresa, que no tienen contacto directo con el cliente pero son necesarias para que la organización funcione.

Incluye: Gestión de recursos humanos, Nómina y selección de personal, Facturación, Finanzas, Contabilidad, Gestión de compras, Logística, Análisis de negocio, CRM

BPOS comunes:

- BPO de Customer Contact: orientado a la atención del cliente.
- BPO de Administración y Finanzas: Estas áreas suelen tercerizarse porque requieren conocimientos específicos y permiten reducir costos administrativos.
- BPO de Recursos Humanos: La diapositiva destaca especialmente el servicio de reclutamiento.
- BPO de Servicios de Gestión de la Información: Consiste en controlar el flujo de información y los medios de soporte (hardware y software), además de todo lo relacionado con tecnologías de la información y comunicación.

Las empresas suelen tercerizar para:

- Concentrarse en las actividades principales del negocio
- Reducir costos
- Convertir costos fijos en costos variables
- Nuevos motivos

También aparecen nuevas razones:

- Necesidad de servicios dinámicos y adaptables
- Disponibilidad de recursos y tecnología especializada
- Costos más previsibles
- Mejora de la eficiencia global de la empresa, y no solo de una tarea puntual

Factores condicionales para aplicar BPO

Ambiente estratégico: Para aplicar BPO, toda la organización debe conocer

- La estrategia
- La visión
- La misión
- Los objetivos
- Las metas de la empresa

Ambiente organizacional: La tercerización produce cambios en:

- La estructura funcional
- La delegación de autoridad
- Los organigramas

- Las normas y métodos de trabajo

Ambiente económico: La tercerización modifica la estructura de costos de la empresa, por lo que hay que evaluar si realmente conviene económicamente.

- Costos de producción
- Costos de distribución
- Costos administrativos
- Inversiones y activos

Ambiente social: La tercerización puede generar cambios importantes en el empleo y en la estructura interna de la empresa.

Apertura de nuevos negocios: La tercerización puede generar nuevas empresas y oportunidades de negocio dentro del sector de servicios.

Revisión de funciones: La empresa debe redefinir funciones y responsabilidades. La alta gerencia debe analizar cómo cambian las estructuras internas.

Actividades que NO deben tercerizar: Estas actividades son demasiado importantes para el funcionamiento y control de la empresa, por lo que perderlas podría generar dependencia o pérdida de control.

SLA (Service Level Agreement): Es un contrato o acuerdo que describe el nivel de servicio que un cliente espera de su proveedor. En español también se llama Acuerdo de Nivel de Servicio (ANS).

El SLA sirve para establecer indicadores medibles que permitan regular el servicio prestado y asegurar que se cumplan las expectativas del cliente. Por ejemplo, puede definir:

- Tiempo máximo de respuesta
- Tiempo máximo de resolución
- Disponibilidad del servicio
- Cantidad de incidentes permitidos

SLA de servicio: Es un SLA estándar que se aplica a todos los clientes que contratan un mismo servicio.

SLA basado en el cliente: Se aplica a todos los servicios contratados por un mismo cliente o grupo de clientes. Por ejemplo, un cliente importante puede tener prioridad sobre otros, aunque use el mismo servicio. También puede existir un SLA especial para el departamento de finanzas de una empresa.

SLA multinivel: Combina varios SLA al mismo tiempo:

- SLA corporativo
- SLA por cliente
- SLA por servicio

Ejemplo: una empresa puede tener un SLA general para todos los empleados, otro más exigente para directivos y otro especial para un servicio crítico.

Los SLA aportan varias ventajas, permite controlar la calidad del servicio de manera objetiva.

- Mejoran el servicio prestado
- Reducen el tiempo de resolución de incidentes
- Facilitan la comunicación entre cliente y proveedor
- Generan un acuerdo mutuo y claro
- Evitan disputas porque todo queda por escrito
- Permiten medir objetivamente si el proveedor cumple
- Proporcionan reglas y estándares de gestión

Software Factory / Factoría de Software: Es una empresa o grupo especializado en el desarrollo y mantenimiento de sistemas de información para un cliente, siguiendo requisitos previamente definidos. Trabaja como una “fábrica” de software, utilizando procesos repetibles y organizados. Se basa en cuatro principios:

- Agilidad
- Automatización
- Analítica de la información
- Seguridad

Además, se caracteriza por:

- Actividades de desarrollo previsibles
- Reutilización de componentes
- Reducción de trabajo repetitivo
- Enfoque en segmentos específicos del mercado
- Mayor calidad en los resultados

Beneficios de una Software Factory

- Productividad: Muchas tareas de desarrollo pueden simplificarse o automatizarse. Además, permite que la empresa se concentre en su actividad principal y deje el desarrollo en manos de especialistas.
- Automatización: Reduce errores, libera recursos y aumenta la velocidad de respuesta. La software factory trabaja con procesos estandarizados y personal especializado.
- Ahorro: “Industrializar” el desarrollo de software reduce costos y acortar el ciclo de desarrollo en comparación con un equipo interno tradicional.
- Calidad: El software desarrollado puede verificarse antes de implementarse y se aplican mejores prácticas durante todo el proceso. La seguridad también se incorpora desde el inicio hasta la entrega final.

¿Por qué contratar una Software Factory?

- La empresa no tiene un software existente que cubra sus necesidades.
- El equipo interno está demasiado ocupado o no tiene los conocimientos necesarios.
- El desarrollo interno es más costoso que contratar una software factory.

PREGUNTAS

1. Que es tercerizacion
2. Que es bpo
3. Cuales son los factores que condicionan al bpo desarrolle 3
4. Describa y ejemplifique tipos de SLA
5. Veneficios y desventajas de sw factory

UNIDAD 4 — GESTIÓN DE CONFIGURACIONES

Gestión de configuraciones: Consiste en administrar las configuraciones del sistema para asegurar coherencia e integridad en todos los entornos. Supervisa y controla los cambios en los metadatos de configuración de los sistemas de sw. Incluye

- Control de versiones
- Gestión de cambios
- Documentación

Integración continua: Práctica en la que los desarrolladores integren sus cambios en el repositorio compartido varias veces al día. Cada integración desencadena automáticamente la ejecución de pruebas automatizadas para verificar errores de forma temprana, esto ayuda a reducir el riesgo de errores y conflictos entre los distintos códigos.

Ejemplos de pruebas automáticas en CI

- Verificar que el login funcione correctamente.
- Comprobar que una compra se guarde en la base de datos.
- Revisar que el sistema genere una factura luego de una venta.
- Validar que un formulario web muestre “Registro exitoso”.
- Confirmar que dos servicios puedan comunicarse correctamente.

Entrega Continua: Es una práctica en la que cada cambio que pasa satisfactoriamente por el proceso de integración continua queda listo para ser desplegado en producción. El software se mantiene siempre en un estado desplegable.

Si un equipo corrige un error en una aplicación y esa corrección supera todas las pruebas automáticas, la nueva versión queda lista para ser instalada en producción cuando la empresa lo decida.

Despliegue Continuo: Va un paso más allá que la entrega continua. Además de pasar las pruebas automatizadas, los cambios se despliegan automáticamente en producción sin intervención humana. El objetivo es entregar los cambios a los usuarios finales tan pronto como estén listos, acelerando el tiempo de entrega y aumentando la eficiencia del equipo de desarrollo.

Diferencia entre Entrega Continua y Despliegue Continuo

En Entrega Continua el software queda listo, pero alguien decide cuándo desplegarlo.

En Despliegue Continuo el sistema se despliega automáticamente apenas aprueba todas las pruebas.

Gestión de la configuración: ayuda a los equipos de ingeniería a crear sistemas sólidos y estables mediante el uso de herramientas que gestionan y supervisan automáticamente las actualizaciones de los datos de configuración.

Sin una correcta gestión de configuración, pueden aparecer problemas como:

- Distintas versiones de configuración entre ambientes
- Contraseñas o claves incorrectas
- Errores al desplegar
- Sistemas que funcionan en desarrollo pero no en producción

Ejemplo: Arquitectura de Microservicios

En una arquitectura de microservicios, cada servicio utiliza metadatos de configuración para registrarse e inicializarse. Si uno de estos datos está mal configurado, el microservicio puede dejar de funcionar o no poder comunicarse con el resto del sistema. Algunos ejemplos son:

- Cantidad de CPU y RAM asignada
- Dirección de otro servicio o base de datos
- Usuario y contraseña de acceso
- Claves de cifrado o tokens

La **metodología ágil** también se aplica al trabajo de configuración. Los cambios de configuración pueden clasificarse, priorizarse y planificarse dentro de los sprints.

Ejemplos de tareas de configuración en un sprint

Actualizar certificados SSL de producción
Agregar un nuevo endpoint de base de datos
Cambiar contraseñas de servicios de correo
Agregar claves API para integrar un nuevo sistema externo

Plan de Gestión de la Configuración de IT: Un buen plan de gestión de configuración debe incluir varios elementos.

1. **Propósito u objetivo:** Describe cómo se llevará a cabo la gestión de la configuración durante todo el ciclo de vida del servicio y qué se espera mejorar.
2. **Alcance** Define qué servicios o prestaciones de IT estarán bajo control de la gestión de configuración.
Ejemplo: Puede incluir servidores, bases de datos, aplicaciones y redes.
3. **Convenciones de nomenclatura:** Cada elemento de configuración (CI) debe tener un nombre o identificador único para poder identificarlo rápidamente cuando haya un incidente o un cambio.
Ejemplo:
SERV-PROD-01
DB-DESARROLLO-03
Línea de Base (Baseline)
4. **Línea de base:** Es la identificación exacta de qué compone un servicio crítico y sus dependencias clave. Sirve como punto de referencia estable.
Ejemplo: La línea de base de una aplicación puede incluir:
Versión del software
Servidor utilizado
Base de datos
Configuración de red
5. **Relación con la Gestión de Cambios:** La gestión de configuración debe estar alineada con el proceso de gestión de cambios para asegurar que cualquier actualización quede reflejada correctamente y coincida con el entorno real de producción.
6. **Contabilidad del Estado:** Consiste en registrar con precisión en qué etapa del ciclo de vida se encuentra cada elemento de configuración.
Ejemplo: Un servidor puede estar:
En desarrollo
En prueba
En producción
Retirado
7. **Actividades de Verificación:** Son comprobaciones rutinarias para asegurar que la información registrada coincide con la realidad.
Ejemplo: Verificar el número de serie de una PC cuando un usuario registra un incidente.
8. **Actividades de Auditoría:** Las auditorías consisten en revisar periódicamente las líneas de base y los elementos de configuración para detectar inconsistencias, generar reportes y aprender de los problemas encontrados. Además, debe existir un plan para comunicar los resultados de las auditorías a los interesados y a la dirección.
9. **Funciones y Responsabilidades:** La gestión de la configuración requiere definir claramente quién hace cada tarea. Como es una actividad muy orientada al detalle, deben asignarse roles y responsabilidades específicos.
Se recomienda usar un diagrama RACI para indicar:
Quién realiza la tarea
Quién aprueba
Quién debe ser consultado
Quién debe ser informado

Gestión de Control de Versionados (Repositorio Git): Se utiliza para gestionar los datos de configuración. Git permite el control de versiones y hace que el repositorio actúe como fuente de información oficial del sistema. El

control de versiones de la configuración permite revertir o “deshacer” una configuración para volver rápidamente a un último estado estable conocido. Esto evita modificaciones inesperadas o errores difíciles de corregir. Se pueden cifrar algunas versiones que tengan código que no tiene que ver todo el mundo

Gestión tradicional vs DevOps: Antes, la administración de sistemas se hacía manualmente mediante scripts y procesos ad-hoc. Los datos de configuración podían pasarse por alto, aunque eran fundamentales para el funcionamiento del sistema. Con DevOps, la gestión de la configuración pasa a ser una parte central y automatizada del proceso.

Bóveda de claves: Es un lugar donde se guardan todas las contraseñas

Configuración en DevOps: La configuración de DevOps representa la evolución y automatización de la administración de sistemas. DevOps traslada parte de la responsabilidad de administrar la infraestructura desde los administradores de sistemas hacia los equipos de desarrollo. Los desarrolladores pueden solicitar y aprovisionar recursos bajo demanda, evitando cuellos de botella producidos por depender de otro equipo. Ejemplo: Un desarrollador puede crear automáticamente un nuevo servidor o una nueva base de datos sin esperar que el área de infraestructura lo haga manualmente.

Infraestructura como Código (Infrastructure as Code – IaC): Es un enfoque en el que la infraestructura se define mediante archivos de configuración legibles por personas y máquinas, por ejemplo archivos YAML. Esto ayuda a agilizar el proceso, de repetibilidad, menor cantidad de errores, posibilidad de versionar la infraestructura, etc. Las plataformas cloud permiten especificar:

- Recursos de hardware
- Conexiones de red
- Servidores
- Bases de datos
- Reglas de acceso

La gestión de la configuración en CI y CD utiliza flujos de trabajo de revisión de código basados en solicitudes de incorporación de cambios (pull requests) para automatizar la implementación de cambios de configuración.

Seguimiento de Versiones: Es necesario mantener un control estricto de las versiones del software, asegurando que cada cambio esté asociado a una versión específica y que se puedan identificar las diferencias entre versiones

Control de Acceso: Se debe establecer un sistema de control de acceso para garantizar que solo las personas autorizadas puedan proponer, revisar, aprobar e implementar cambios en el software. No todo el equipo de IT tiene que tener acceso a todo lo que se desarrolla en el equipo, hay proyectos que son sensibles, y cada profesional ve en lo que tiene que trabajar.

Evaluación de cambios:

Análisis de Impacto: Cada cambio propuesto debe ser evaluado para determinar su impacto en el sistema, incluyendo posibles efectos secundarios en funcionalidades existentes y su compatibilidad con otros componentes del software

Pruebas de Calidad: Antes de aprobar un cambio, se deben realizar pruebas exhaustivas para verificar que el cambio funcione

Revisión por Pares: Los cambios propuestos deben ser revisados por otros miembros del equipo o expertos relevantes para garantizar su calidad y conformidad con los estándares establecidos

Aprobación de cambios:

Autorización: Pueden ser líderes del equipo de desarrollo, arquitectos de software o responsables de calidad, dependiendo de la naturaleza y el alcance del cambio

PREGUNTAS:

1. Desarrolle el concepto de gestion de las configuraciones en it
2. - Que es integracion continua, explique y ejemplifique
3. - Que es entrega continua y despliegue continuo, explique y ejemplifique
4. - Por que es importante la gestion de configuracion, desarrolle un ejemplo
5. - Desarrolle tecnologia vs metodologia DevOps
6. - Que es IAC, explique
7. - Desarrolle gestion de la configuracion en DevOps
8. - Desarrolle gestion de la configuracion en la Ci y Cd
9. - Desarrolle gestion de la configuracion en metodologia agile
10. - Para que se utiliza el plan de gestion de las configuraciones it, mencioné al menos 3 pasos
11. - Cuales son las principales actividades de la gestion de configuraciones. Desarrolle al menos 2
12. - Explique como implementar la gestion de las configuraciones
13. - Que implica requerimientos para el control de cambios
14. - Que implica la evaluacion de cambios
15. - Que implica la aprobacion del cambio

UNIDAD 5

Políticas de Gobernanza de Datos:

Política de Calidad de Datos: Define los estándares para la precisión, integridad, consistencia y confiabilidad de los datos.

Política de Seguridad de Datos: Establece los procedimientos para proteger los datos contra accesos no autorizados, pérdida o corrupción.

Política de Privacidad de Datos: Garantiza el cumplimiento de las leyes y regulaciones relacionadas con la privacidad de los datos.

Política de Retención de Datos: Detalla cuánto tiempo se deben conservar los diferentes tipos de datos y los procedimientos para su eliminación segura.

Normativas y Estándares

Normativas Internas: Directrices específicas a la organización para la gestión y uso de los datos. Estas pueden incluir cómo se deben documentar los metadatos, las mejores prácticas para la gestión de datos maestros y las reglas para la integración de datos.

Estándares de la Industria: Adopción de estándares internacionales y de la industria, como ISO 27001 para la seguridad de la información, o COBIT para la gobernanza y gestión de TI.

Roles y Responsabilidades

Data Owners (Propietarios de Datos): Responsables de la gestión y calidad de los datos dentro de su dominio.

Data Governance Council (Consejo de Gobernanza de Datos): Grupo de alto nivel que supervisa la estrategia de gobernanza de datos y resuelve conflictos.

Data Stewards (Custodios de Datos): Encargados de implementar las políticas de gobernanza de datos y asegurar el cumplimiento de los estándares.

Procesos y Procedimientos

Gestión de Calidad de Datos: Procesos para monitorear, reportar y mejorar continuamente la calidad de los datos.

Manejo de Incidentes de Datos: Procedimientos para identificar, reportar y resolver incidentes relacionados con la seguridad o calidad de los datos.

Auditoría y Cumplimiento: Procesos para asegurar que las políticas y normativas se sigan y para realizar auditorías periódicas.

Tecnologías de Soporte

Herramientas de Gestión de Datos: Software para la gestión de metadatos, calidad de datos, integración de datos y seguridad.

Plataformas de Gobernanza de Datos: Soluciones tecnológicas que facilitan la implementación y el seguimiento de políticas y procedimientos de gobernanza de datos.

Implementación de Políticas y Normativas: Para implementar eficazmente un marco de gobernanza de datos, es crucial:

1. **Desarrollar y Documentar Políticas y Normativas**
2. **Capacitación y Concientización:** Asegurar que todos los empleados comprendan la importancia de la gobernanza de datos y estén familiarizados con las políticas y procedimientos.
3. **Monitoreo y Revisión Continua:** Establecer mecanismos para monitorear el cumplimiento y revisar regularmente las políticas y normativas para adaptarse a cambios en el entorno regulatorio o en las necesidades de la organización.

Gestión de Riesgos: La gestión de riesgos en la gobernanza de datos implica identificar, evaluar y mitigar los riesgos asociados con la gestión de datos. Esto incluye riesgos relacionados con la seguridad, privacidad, calidad y disponibilidad de los datos.

Identificación de Riesgos

Análisis de Amenazas: Identificar amenazas potenciales que puedan comprometer los datos, como ataques cibernéticos, errores humanos, desastres naturales, etc.

Evaluación de Vulnerabilidades: Analizar las debilidades en los sistemas y procesos que podrían ser explotadas por las amenazas.

Evaluación de Riesgos

Evaluación de Impacto: Determinar las consecuencias potenciales de cada riesgo identificado.

Probabilidad de Ocurrencia: Estimar la probabilidad de que cada riesgo ocurra.

Mitigación de Riesgos

Medidas de Control: Implementar controles y procedimientos para reducir la probabilidad y el impacto de los riesgos. Esto puede incluir cifrado de datos, controles de acceso, copias de seguridad regulares, etc.

Planes de Contingencia: Desarrollar planes de respuesta y recuperación ante incidentes para minimizar el impacto de los riesgos que se materialicen.

Aseguramiento del Cumplimiento de Regulaciones y Estándares:

Identificación de Regulaciones y Estándares Aplicables

Regulaciones Locales e Internacionales: Identificar las leyes y regulaciones que afectan a la organización, como el GDPR en Europa, CCPA en California, HIPAA en el sector de salud en EE.UU., entre otros.

Estándares de la Industria: Adoptar estándares reconocidos, como ISO 27001 para la gestión de la seguridad de la información, y COBIT para la gobernanza y gestión de TI.

Implementación de Políticas y Procedimientos de Cumplimiento

Políticas de Cumplimiento: Desarrollar y documentar políticas claras que aseguren el cumplimiento con las regulaciones y estándares aplicables.

Capacitación y Concientización: Asegurar que todos los empleados comprendan las políticas y sepan cómo cumplir con ellas.

Diferencia entre la política y el procedimiento: la política es la postura oficial de la empresa sobre cómo se debe manejar un problema, compuesta de reglas, con más foco en lo que no se debe hacer, y su alcance suele ser de toda la empresa. El procedimiento no necesariamente es una postura oficial ni una mejor práctica, está compuesto de pasos, con más enfoque en lo que se debe hacer, y su alcance se puede limitar a una tarea o actividad específica.

Monitoreo y Auditoría

Auditorías Regulares: Realizar auditorías periódicas para verificar el cumplimiento de las políticas y procedimientos establecidos.

Monitoreo Continuo: Utilizar herramientas de monitoreo para detectar y corregir incumplimientos en tiempo real.

Reporte y Mejora Continua

Reporte de Cumplimiento: Mantener registros detallados y reportar el estado de cumplimiento a las partes interesadas y autoridades reguladoras según sea necesario.

Revisión y Mejora Continua: Evaluar y actualizar regularmente las políticas y procedimientos de cumplimiento para adaptarse a cambios en las regulaciones y en el entorno operativo.

Control Self Assessments (CSA): Son procesos en los cuales los empleados de una organización evalúan la efectividad de los controles internos y la conformidad con las políticas y procedimientos establecidos. Esta autoevaluación permite identificar debilidades, riesgos y áreas de mejora desde una perspectiva interna.

Beneficios de los CSA

Identificación Temprana de Problemas: Permiten detectar riesgos y deficiencias en los controles antes de que ocurran incidentes graves.

Mejora Continua: Facilitan la identificación de oportunidades para mejorar los procesos y controles internos.

Mayor Responsabilidad: Fomentan la responsabilidad y la conciencia sobre la gestión de riesgos y el cumplimiento entre los empleados.

Reducción de Costos: Al identificar y corregir problemas tempranamente, se pueden reducir los costos asociados con la mitigación de riesgos y la remediación de incidentes.

Implementación de los CSA

Preparación y Planificación: Definir el Alcance; Establecer Criterios.

Ejecución de la Autoevaluación: Recolección de Información; Evaluación de Controles.

Análisis de Resultados: Identificación de Debilidades; Evaluación de Impacto.

Desarrollo de Planes de Acción: Recomendaciones; Planes de Mejora.

Monitoreo y Seguimiento: Implementación de Mejoras; Monitoreo Continuo.

Tipos de Copias de Seguridad

Copia de Seguridad Completa (Full Backup): implica copiar todos los datos seleccionados a un medio de almacenamiento. Este tipo de backup es el más básico y garantiza que todos los datos estén respaldados en un solo conjunto.

Ventajas: Recuperación rápida y sencilla ya que todos los datos están en un solo lugar. Es fácil de gestionar y entender.

Desventajas: Consume mucho tiempo y espacio de almacenamiento. No es eficiente para datos que cambian poco.

Copia de Seguridad Incremental (Incremental Backup): solo copia los datos que han cambiado desde la última copia de seguridad, ya sea completa o incremental.

Ventajas: Requiere menos tiempo y espacio de almacenamiento que una copia completa. Es eficiente en términos de recursos.

Desventajas: La restauración puede ser lenta, ya que requiere la última copia completa más todas las copias incrementales subsecuentes. Es más complejo de gestionar.

Copia de Seguridad Diferencial (Differential Backup): Copia todos los datos que han cambiado desde la última copia de seguridad completa. A diferencia de la incremental, no se basa en la última copia incremental.

Ventajas: Restauración más rápida que una incremental, ya que solo se necesita la última copia completa y la última copia diferencial. Balance entre tiempo de copia y facilidad de restauración.

Desventajas: A medida que pasa el tiempo desde la última copia completa, las copias diferenciales pueden crecer en tamaño, consumiendo más espacio y tiempo.

Copia de Seguridad en Espejo (Mirror Backup): es similar a una copia completa, pero los archivos no se comprimen ni se agrupan en un solo archivo. Es una réplica exacta de los datos originales.

Ventajas: Los archivos están listos para su uso inmediato. Rápida recuperación de archivos individuales.

Desventajas: No incluye versiones anteriores de los archivos. Consume más espacio de almacenamiento y no tiene compresión.

Copia de Seguridad Incremental Reversa (Reverse Incremental Backup): Este método crea una copia de seguridad completa y luego aplica los cambios incrementales al archivo completo para mantenerlo actualizado.

Ventajas: Rápida restauración, ya que siempre se mantiene una copia completa y actualizada.

Ofrece beneficios de backups incrementales con restauraciones más rápidas.

Desventajas: Puede ser complejo de implementar y gestionar. Requiere más procesamiento durante la copia de seguridad.

Copia de Seguridad en la Nube (Cloud Backup): Copiar datos a un servicio de almacenamiento en la nube ofrecido por proveedores como AWS, Google Cloud, o Microsoft Azure.

Ventajas: Acceso remoto y recuperación desde cualquier lugar con conexión a Internet. Escalabilidad y reducción de costos de infraestructura local.

Desventajas: Depende de la velocidad y estabilidad de la conexión a Internet. Preocupaciones sobre la seguridad y privacidad de los datos en la nube.

Copia de Seguridad Continua (Continuous Data Protection, CDP): Realiza copias de seguridad de los datos continuamente cada vez que ocurre un cambio. Esto permite la recuperación de datos en cualquier punto en el tiempo.

Ventajas: Ofrece la capacidad de restaurar datos a cualquier punto específico. Minimiza la pérdida de datos.

Desventajas: Requiere mucho almacenamiento y recursos. Puede ser complejo de configurar y gestionar.

BCP y DRP: Son componentes críticos para garantizar que una organización pueda sobrevivir y seguir operando en caso de interrupciones graves. Ayudan a preparar a la organización para diversas contingencias y asegurar que los procesos esenciales se mantengan operativos.

Pasos para Realizar una Simulación de Continuidad del Negocio

1. **Definición del Alcance y Objetivos:** Establecer claramente qué aspectos del DRP y BCP se van a probar. Definir los objetivos específicos de la simulación, como tiempos de recuperación, eficacia de las comunicaciones, etc.
2. **Desarrollo del Escenario:** Crear un escenario realista que represente una interrupción significativa, como un desastre natural, un ciberataque o una falla de infraestructura. Incluir detalles específicos sobre el tipo de incidente, su impacto esperado y las condiciones iniciales.
3. **Asignación de Roles y Responsabilidades:** Designar a los participantes y definir sus roles durante la simulación. Asegurar que cada miembro del equipo entienda sus responsabilidades y tareas durante el ejercicio.
4. **Ejecución de la Simulación:** Llevar a cabo la simulación siguiendo el escenario planificado. Monitorear y registrar las acciones tomadas y el tiempo requerido para completar cada tarea.
5. **Evaluación y Análisis de Resultados:** Revisar los resultados de la simulación para identificar éxitos y áreas de mejora. Evaluar si se cumplieron los objetivos y qué tan bien se siguieron los planes.
6. **Revisión y Actualización de los Planes:** Actualizar el DRP y BCP según los hallazgos de la simulación. Incorporar las lecciones aprendidas y ajustar las estrategias y procedimientos.

Plan de Recuperación de Desastres (DRP): se centra en la restauración de sistemas tecnológicos y datos críticos tras una interrupción. Este plan es crucial para minimizar el tiempo de inactividad y asegurar que la organización pueda reanudar sus operaciones normales lo antes posible.

Componentes Clave de un DRP:

1. Evaluación de Riesgos
2. Inventario de Recursos
3. Estrategias de Recuperación
4. Plan de Comunicación
5. Procedimientos de Recuperación

6. Pruebas y Mantenimiento

Plan de Continuidad de Operaciones (BCP): Se enfoca en mantener todas las operaciones esenciales de la organización durante y después de una interrupción. Este plan abarca más allá de la tecnología e incluye procesos, personal y otras funciones críticas.

Componentes Clave de un BCP:

1. Análisis de Impacto en el Negocio (BIA)
2. Estrategias de Continuidad
3. Planes de Acción
4. Plan de Comunicación
5. Capacitación y Concientización
6. Pruebas y Revisión

PREGUNTAS:

1. ¿Qué es gobernanza de software?
2. ¿Cuál es la diferencia entre proceso, política y procedimiento?
3. ¿Qué es gobierno de datos?
4. Explique y ejemplifique políticas de gobernanza de datos.
5. ¿Cuál es la diferencia entre privacidad de datos vs retención de datos?
6. Cuáles son los estándares de la industria?
7. Diferencia entre Data Owner, Data Stewards, Data Governors, Data Council.
8. Desarrolle auditoría y cumplimiento.
9. Describa el proceso de gestión de datos. Explique al menos 4 etapas.
- 10.Cuál es la relación de Data governance vs analítics y machine learning.
11. Desarrolle gestión de riesgos en ingeniería de software (no admin de proyectos).
12. Descripción de evaluación de vulnerabilidades.
13. ¿Qué son las estrategias de mitigación de riesgo? Ejemplifique.
14. ¿Cuál es la diferencia entre auditorías regulares y monitoreo continuo?
15. ¿Qué es PDCA?
16. ¿Qué es CSA?
17. Cómo se implementa un CSA?
18. Desarrolle y ejemplifique tipos de copias de seguridad. (3 principales y varios complementarios)
19. ¿Qué es DRP y BCP? Ventajas y desventajas de c/u.
20. Mencione y describa al menos 4 pasos para realizar una simulación de continuidad de negocio.
21. Desarrolle al menos 4 etapas del DRP.
22. Desarrolle al menos 4 etapas del BCP.

UNIDAD 6

Principios Básicos de Seguridad en el Desarrollo de Software: Son fundamentales para crear aplicaciones robustas y proteger los datos de los usuarios.

Validación de Entrada: Asegúrate de que todas las entradas del usuario sean validadas y limpiadas para evitar inyecciones de código (como SQL injection, XSS, etc.). Cross-Site Scripting (XSS) es un tipo de vulnerabilidad de seguridad que permite a un atacante inyectar scripts maliciosos en páginas web vistas por otros usuarios.

Gestión de Autenticación y Autorización

Autenticación Segura: Implementa métodos de autenticación fuertes, como autenticación multifactor (MFA).

Control de Acceso: Asegúrate de que los usuarios solo puedan acceder a los recursos para los que están autorizados, siguiendo el principio de privilegio mínimo.

Cifrado de Datos

Datos en Tránsito: Usa protocolos seguros como HTTPS para proteger los datos en tránsito.

Datos en Reposo: Cifra los datos almacenados para protegerlos en caso de que el sistema sea comprometido.

Gestión de Sesiones

Sesiones Seguras: Genera identificadores de sesión únicos y difíciles de adivinar. Usa cookies seguras y establece tiempos de expiración razonables.

Protección contra Secuestro de Sesión: Implementa medidas para evitar el secuestro de sesión, como la regeneración de ID de sesión después de la autenticación.

Principio de Privilegio Mínimo

Acceso Restringido: Los usuarios y procesos deben tener solo los permisos necesarios para realizar sus funciones.

Revisión Regular: Realiza auditorías y revisiones periódicas de los permisos de los usuarios.

Defensa en Profundidad

Capas de Seguridad: Implementa múltiples capas de seguridad (firewalls, IDS/IPS, controles de acceso, etc.) para proteger el sistema.

Redundancia: Cada capa debe ser capaz de mitigar el impacto de una falla en otra capa.

IDS/IPS: Sistemas de Detección y Prevención de Intrusiones: Los Sistemas de Detección de Intrusiones (IDS) y los Sistemas de Prevención de Intrusiones (IPS) son tecnologías esenciales para la seguridad de redes y sistemas. Ambos se utilizan para identificar y mitigar actividades maliciosas, pero funcionan de manera diferente.

IDS (Intrusion Detection System): Un IDS es una herramienta de monitoreo que analiza el tráfico de red y los sistemas en busca de actividades sospechosas o maliciosas. No toma medidas directas para detener las intrusiones, pero alerta a los administradores para que tomen acción.

IPS (Intrusion Prevention System): Un IPS no solo detecta actividades sospechosas como un IDS, sino que también puede tomar medidas automatizadas para prevenir intrusiones, como bloquear el tráfico malicioso.

Registro y Monitoreo

Auditoría y Logs: Registra actividades importantes del sistema para detectar y responder a incidentes de seguridad.

Monitoreo Activo: Implementa sistemas de monitoreo para detectar comportamientos anómalos y responder rápidamente a posibles ataques.

Pruebas de Seguridad

Pruebas de penetración: Realiza pruebas de penetración periódicas para identificar y corregir vulnerabilidades.

Análisis de Código: Utiliza herramientas de análisis estático y dinámico para detectar posibles problemas de seguridad en el código.

Actualizaciones y Parches

Gestión de Parches: Mantén el software y las dependencias actualizados con los últimos parches de seguridad.

Automatización: Utiliza herramientas de automatización para aplicar parches de manera oportuna.

Concienciación y Formación

Capacitación del Personal: Educa a los desarrolladores y empleados sobre las mejores prácticas de seguridad y amenazas emergentes.

Cultura de Seguridad: Fomenta una cultura de seguridad dentro de la organización, donde la seguridad sea una prioridad en todas las fases del desarrollo.

Seguridad en el Ciclo de Vida del Desarrollo (SSDLC): Integra prácticas de seguridad en cada fase del ciclo de vida del desarrollo de software. Esto garantiza que los productos finales sean más seguros y resistentes a ataques.

Requisitos de SSDLC

Requisitos de Seguridad: Define claramente los requisitos de seguridad desde el principio. Esto incluye la protección de datos, el cumplimiento normativo y la gestión de accesos.

Evaluación de Riesgos: Realiza evaluaciones de riesgos para identificar amenazas potenciales y su impacto.

Diseño de SSDLC

Revisión de Arquitectura: Realiza revisiones de arquitectura para identificar y mitigar posibles vulnerabilidades.

Modelado de Amenazas: Usa el modelado de amenazas para identificar y documentar amenazas potenciales y las medidas para mitigarlas.

Principios de Diseño Seguro: Aplica principios de diseño seguro, como el principio de privilegio mínimo, defensa en profundidad y diseño de componentes con interfaces seguras.

Implementación de SSDLC

Codificación Segura: Sigue las mejores prácticas de codificación segura y estándares de la industria, como OWASP Secure Coding Practices.

Revisiones de Código: Realiza revisiones de código (code reviews) para detectar vulnerabilidades y malas prácticas.

Pruebas de SSDLC

Pruebas de Seguridad: Incluye pruebas de seguridad como parte de las pruebas funcionales. Esto puede incluir pruebas de penetración, fuzzing y análisis de vulnerabilidades.

Automatización de Pruebas: Integra pruebas de seguridad automatizadas en el pipeline de integración continua/entrega continua (CI/CD).

Revisión de Dependencias: Asegúrate de que las bibliotecas y dependencias utilizadas no tengan vulnerabilidades conocidas mediante herramientas de análisis de dependencias.

El fuzzing, también conocido como "fuzz testing", es una técnica de prueba automatizada utilizada para identificar vulnerabilidades de seguridad en software, aplicaciones y sistemas. Consiste en ingresar datos

aleatorios, inesperados o malformados en un programa para observar cómo maneja estas entradas y detectar posibles fallos o comportamientos no deseados.

Despliegue de SSDLC

Configuración Segura: Asegúrate de que los entornos de producción estén configurados de manera segura, incluyendo la configuración de servidores, bases de datos y redes.

Gestión de Claves y Certificados: Implementa prácticas seguras para la gestión de claves y certificados.

Contenedores y DevOps: Si usas contenedores, asegura las imágenes de los contenedores y utiliza herramientas como Kubernetes para gestionar la seguridad en la orquestación de contenedores.

Implementación de herramientas de gestión de secretos como HashiCorp Vault. Es una herramienta diseñada para gestionar secretos y proteger datos sensibles. Facilita el almacenamiento, acceso y gestión segura de secretos (como tokens, contraseñas, certificados, y claves API) mediante una interfaz unificada y políticas de control de acceso.

Mantenimiento

Actualizaciones y Parches: Aplica parches y actualizaciones de seguridad de manera oportuna para mitigar vulnerabilidades.

Monitoreo y Detección de Incidentes: Implementa sistemas de monitoreo para detectar y responder a incidentes de seguridad.

Revisiones de Seguridad Periódicas: Realiza revisiones de seguridad periódicas para evaluar la efectividad de las medidas de seguridad implementadas.

Autenticación: Es el proceso de verificar la identidad de un usuario o sistema. Es el primer paso para asegurar que el acceso a los sistemas sea legítimo

Métodos de autenticación:

1. **Contraseñas:** El método más común que utiliza una combinación de nombre de usuario y contraseña. Ejemplos: inicios de sesión en sitios web, aplicaciones de correo.
2. **Autenticación de Dos Factores (2FA):** Combina algo que el usuario sabe (contraseña) con algo que el usuario tiene (dispositivo). Ejemplos: SMS, aplicaciones de autenticación (Google Authenticator).
3. **Autenticación Multifactor (MFA):** Extiende 2FA al requerir múltiples formas de verificación. Ejemplos: contraseña + huella dactilar + token físico.
4. **Biometría:** Utiliza características físicas del usuario para la autenticación. Ejemplos: huellas dactilares, reconocimiento facial, escaneo de iris.
5. **Certificados Digitales:** Utiliza infraestructura de clave pública (PKI) para verificar la identidad del usuario. Ejemplos: certificados SSL/TLS para servidores web, certificados personales.
6. **Autenticación Basada en Token:** Utiliza tokens generados por sistemas de autenticación para verificar la identidad. Ejemplos: JSON Web Tokens (JWT), OAuth tokens.

Autorización: Es el proceso de determinar si un usuario autenticado tiene permiso para acceder a un recurso específico o realizar una acción determinada. Esto se hace después de la autenticación.

Métodos de autorización:

1. **Listas de Control de Acceso (ACL):** Define permisos específicos para cada usuario o grupo de usuarios para acceder a ciertos recursos. Ejemplos: permisos de archivos en sistemas operativos, firewalls.
2. **Control de Acceso Basado en Roles (RBAC):** Asigna permisos a roles en lugar de a usuarios individuales; los usuarios se asignan a roles específicos. Ejemplos: gestión de permisos en Kubernetes, sistemas de IAM.

3. **Control de Acceso Basado en Atributos (ABAC):** Define permisos basados en atributos del usuario, entorno, recurso y acción. Ejemplos: políticas de acceso en AWS IAM, sistemas avanzados de IAM.
4. **Políticas de Control de Acceso (ACP):** Usa políticas para definir reglas de acceso basadas en varios criterios. Ejemplos: políticas en HashiCorp Vault, políticas de red en software SDN.

Kubernetes (comúnmente abreviado como K8s) es una plataforma de código abierto diseñada para automatizar, desplegar, escalar y administrar aplicaciones en contenedores. Fue desarrollado por Google y luego donado a la Cloud Native Computing Foundation (CNCF). Kubernetes proporciona un entorno de gestión robusto y altamente escalable para contenedores, lo que facilita la implementación y operación de aplicaciones en entornos de nube, locales o híbridos.

Gestión de Privilegios: Se refiere al proceso de otorgar, controlar y revocar los derechos de acceso a los recursos, sistemas y datos en una organización. Esto implica asignar niveles de acceso adecuados a usuarios individuales o roles, garantizando que solo tengan los permisos necesarios para llevar a cabo sus responsabilidades.

Control de Acceso: El control de acceso se refiere a los mecanismos y políticas utilizados para regular y gestionar el acceso a los recursos y datos dentro de un sistema o una red.

Esto implica autenticar a los usuarios, autorizar sus solicitudes y monitorear sus actividades para garantizar que solo accedan a los recursos para los que tienen permiso.

Técnicas para Protección de Datos: Estas técnicas son fundamentales para proteger los datos contra una variedad de amenazas y garantizar la confidencialidad, integridad y disponibilidad de la información en una organización.

1. **Encriptación:** Convierte los datos en un formato ilegible utilizando algoritmos criptográficos y una clave de encriptación.
2. **Autenticación y Control de Acceso:** Verifica la identidad de los usuarios y controla su acceso a los datos basándose en roles y privilegios.
3. **Backups y Recuperación de Desastres:** Realiza copias de seguridad regulares y tiene un plan para restaurar los datos en caso de pérdida o daño.
4. **Monitoreo y Detección de Intrusos:** Supervisa la actividad de la red y los sistemas para identificar y responder a comportamientos sospechosos.
5. **Actualizaciones y Parches:** Mantiene actualizados los sistemas y aplicaciones para protegerse contra vulnerabilidades conocidas.
6. **Segregación de Redes y Segmentación:** Divide la red en segmentos separados y restringe el acceso entre ellos para limitar la propagación de ataques.
7. **Políticas y Procedimientos:** Establece políticas de seguridad y procedimientos operativos para educar a los empleados y garantizar el cumplimiento normativo.
8. **Protección contra Malware:** Utiliza software antivirus y antimalware para detectar y eliminar software malicioso.
9. **Privacidad por Diseño:** Integra la seguridad y la privacidad desde el inicio del diseño de sistemas y aplicaciones.
10. **Auditorías y Pruebas de Penetración:** Realiza auditorías de seguridad y pruebas de penetración para evaluar la eficacia de las medidas de seguridad.

Validación de Datos: Prevención de Ataques como Inyección de Código: La validación de datos es una práctica esencial para prevenir ataques como la inyección de código, que pueden comprometer la seguridad y la integridad de las aplicaciones web y los sistemas de información.

Técnicas de Validación de Datos

1. **Validación de Entrada:**
2. **Filtrado de Entrada:** Filtra y limpia los datos de entrada para eliminar caracteres especiales y potencialmente peligrosos que podrían utilizarse en ataques de inyección.

3. **Validación de Formato:** Verifica que los datos de entrada cumplan con un formato específico (por ejemplo, direcciones de correo electrónico, números de teléfono) para garantizar su integridad y seguridad.
4. **Validación de Longitud:** Limita la longitud de los campos de entrada para evitar ataques de desbordamiento de búfer y otros tipos de ataques basados en la longitud de entrada.
5. **Parámetros de Consulta Seguros:** Utiliza consultas parametrizadas o preparadas en las consultas a la base de datos para separar los datos de los comandos SQL, evitando así la inyección de código SQL.
6. **Codificación de Salida:**
 - a. **Codificación de HTML:** Codifica los datos de salida antes de mostrarlos en una página web para evitar ataques de XSS (Cross-Site Scripting).
 - b. **Codificación de URL:** Codifica los datos de salida que se utilizan en URLs para evitar ataques de inyección de código en las direcciones URL.
7. **Validación del Lado del Servidor y del Cliente:**
 - a. **Validación del Lado del Servidor:** Realiza la validación de datos en el lado del servidor para garantizar que los datos sean seguros y válidos antes de procesarlos o almacenarlos.
 - b. **Validación del Lado del Cliente:** Realiza la validación de datos en el lado del cliente utilizando JavaScript para proporcionar una experiencia de usuario mejorada y prevenir errores antes de enviar los datos al servidor.
8. **Listas de Control de Acceso (ACL):** Utiliza listas de control de acceso (ACL) para restringir el acceso a recursos sensibles y garantizar que solo los usuarios autorizados puedan acceder a ellos.

PREGUNTAS:

1. ¿En qué consiste el desarrollo seguro de software?
2. Mencione al menos 3 principios básicos de seguridad en el desarrollo de software.
3. ¿Qué es XSS en la validación de entrada?
4. Diferencia entre gestión de autenticación y autorización.
5. Diferencia entre datos en tránsito y en reposo.
6. Desarrolle principio de privilegio mínimo.
7. En qué consiste la defensa en profundidad?
8. Diferencia entre IDS vs IPS.
9. ¿Qué son las pruebas de penetración?
10. Desarrolle el enfoque de concientización y formación.
11. Para el ciclo de desarrollo de software, seleccione al menos 3 etapas y explique cómo se involucra la seguridad.
12. ¿Qué es y para qué sirve SSDLC?
13. ¿Para qué se usa OWASP?
14. Desarrolle al menos 4 métodos de autenticación.
15. Desarrolle al menos 2 métodos de autorización.
16. Desarrolle al menos 4 técnicas de protección de datos.
17. Describa posibles técnicas de validación de datos.